

Exercise: Text Analyzer

Overview

In this exercise, you will create a Python module that analyzes text documents. This will give you practice working with Python modules, particularly the `string` and `collections` modules from the standard library. The exercise introduces fundamental concepts from Natural Language Processing (NLP), such as tokenization, frequency analysis, and readability evaluation.

Learning Objectives

- Create and use Python modules
- Work with standard library modules (`string`, `collections`)
- Implement functions with default parameters
- Manipulate and analyze text data
- Evaluate and test code using unit tests

Requirements

1. Implement the `text_analyzer.py` Module

Complete the implementation of the following functions in the skeleton file:

`count_statistics(text)`

- This function should count basic statistics about a text: number of words, number of sentences, and number of characters (excluding spaces).
- Use `split()` to break the text into words.
- Count sentence-ending punctuation marks (`.`, `!`, `?`) to determine the number of sentences.
- If there are no sentence-ending marks but the text is not empty, count as 1 sentence.
- Return a tuple: `(n_words, n_sentences, n_characters)`.

`word_frequency(text, top_n=10, ignore_stopwords=True)`

- This function should analyze the frequency of words in a text.
- The `top_n` parameter determines how many words to return (default: 10).
- The `ignore_stopwords` parameter determines whether to exclude common words like “the”, “is”, “and” (default: True). The `STOPWORDS` set is already provided in the skeleton.
- Before counting, remove punctuation and convert all words to lowercase.
- Use `string.punctuation` with `str.maketrans()` and `str.translate()` to remove punctuation.
- Use `collections.Counter` to count word frequencies.
- Return a list of tuples [(word, count), ...] sorted by frequency in descending order, using `.most_common(top_n)`.

`evaluate_readability(text)`

- This function should evaluate the readability and complexity of a text based on:
 - **Average word length** (longer words indicate more complex vocabulary)
 - **Average sentence length** in words (longer sentences are harder to read)
 - **Vocabulary richness** (ratio of unique words to total words)
- It should return a tuple with:
 - A numeric score (1-5, where 1 is very simple and 5 is very complex)
 - A string describing the level (“Very Simple”, “Simple”, “Moderate”, “Complex”, “Very Complex”)
 - Feedback text explaining the evaluation (it should mention the relevant aspects, e.g. “Words are short and simple. Sentences are short and easy to read.”, “Average word length is high, indicating complex vocabulary. Sentences are long and complex.”)

2. Testing Your Implementation

- Use the provided unit tests in `test_text_analyzer_unit.py` to verify your implementation.
- Run the tests with: `python -m unittest test_text_analyzer_unit.py`
- Your implementation must pass all tests.

Tips

- Study the docstrings in the skeleton file to understand what each function should do.
- For text cleaning, `str.maketrans("", "", string.punctuation)` creates a translation table that removes all punctuation, and `text.translate(table)` applies it.

- `collections.Counter` is very useful for counting occurrences — look up its `.most_common()` method.
- Remember that `set()` can be used to find unique elements in a list.
- Make sure your code handles edge cases (e.g., empty text, text without punctuation).

Submission

Submit your completed `text_analyzer.py` file. Your solution should:

- Pass all the unit tests
- Follow good coding practices (clear variable names, comments where needed)
- Include proper docstrings

Bonus Challenges

If you finish early or want extra practice:

- Add a function `extract_key_sentences(text, n=3)` that returns the `n` most important sentences based on word frequency (extractive summarization)
- Implement a function `detect_language(text)` that guesses whether a text is in English or Italian based on the frequency of language-specific stopwords